

Universidad de las Fuerzas Armadas "ESPE"
Desarrollo de Software Seguro

Implementación de Políticas DevSecOps y Protección de Ramas en GitHub



Integrantes:

- | | |
|--|---|
| <ul style="list-style-type: none">• Caetano Flores• Jordan Guaman | <ul style="list-style-type: none">• Mateo Medranda• Anthony Morales• Leonardo Narváez |
|--|---|

Docente: David Galarza

Fecha: 07/05/2026

El Entorno Permisivo



Inicialmente, los Pull Requests podían integrarse a la rama principal independientemente del estado de los pipelines. El código vulnerable tenía vía libre.

El Entorno Asegurado



Objetivo del Taller: Configurar un entorno automatizado donde sea matemáticamente imposible hacer Merge si el código no supera las pruebas de seguridad e integridad.

[Developer]

[PR]

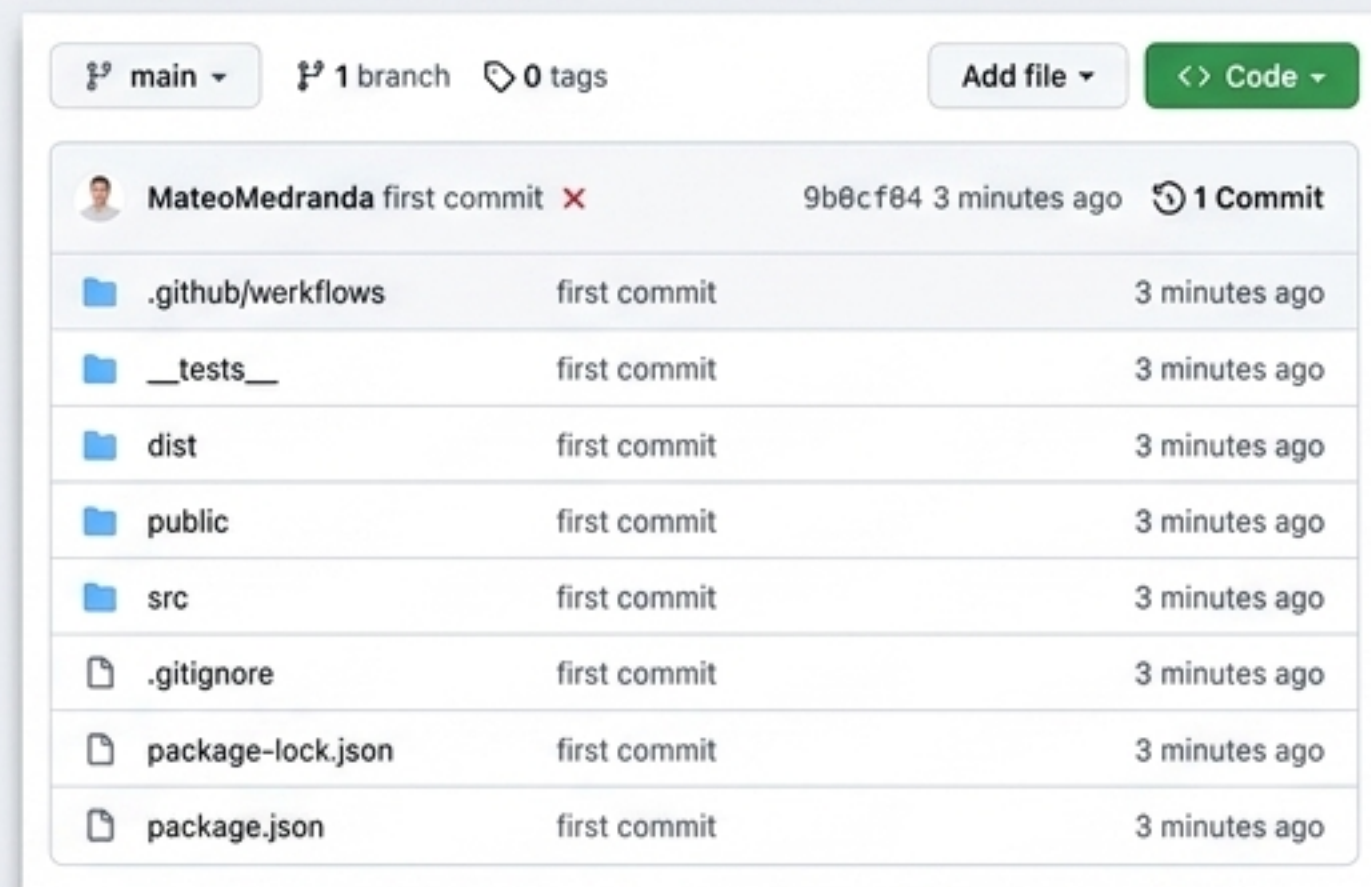
[CI/CD Pipeline]

[Check]

[Main Branch]

El Problema: Código Vulnerable sin Restricciones

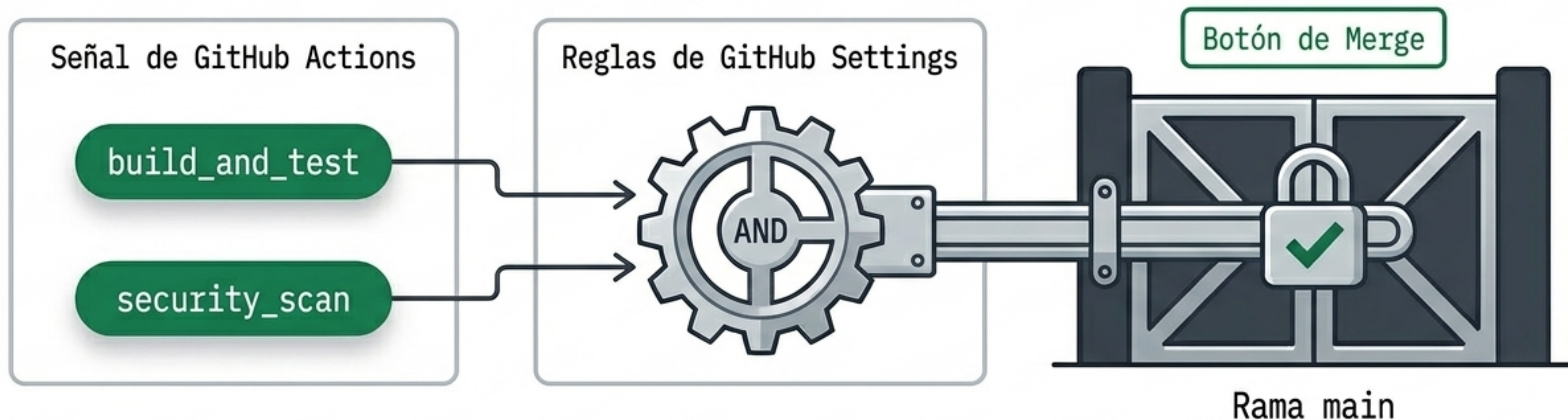
Dimensión	Antes del Taller	Estado Deseado
Integración con Errores	Permitida	Bloqueo Automático
Dependencias Vulnerables	Ignoradas en el Merge	Interceptadas por npm audit
Autorización de Merge	Manual/Subjetiva	Condicionada al Pipeline



La simple existencia de un pipeline no garantiza seguridad si las reglas de la plataforma no obligan a su cumplimiento estricto.

La Solución: Políticas de Protección de Rama

Mecanismo de Protección de Rama



1. Las políticas transforman los flujos de trabajo de opcionales a obligatorios.

2. El Merge funciona ahora como un circuito lógico **AND**: Todas las validaciones deben devolver verdadero (**Success**).

3. Si cualquier nodo **falla**, el engranaje **bloquea** físicamente la compuerta de integración hacia la rama main.

Configuración Crítica

Configuración de Protección de Rama en GitHub

Settings -> Branches -> Branch protection rules

Ruta de acceso a la configuración de la base de código

Branch protection rules

Rule for: main

Protect matching branches

☒ **Require status checks to pass before merging**
Require status checks to pass to be merged to process before merging must now required to permissible matching checks.

La regla de oro:
Condiciona la fusión al éxito de CI/CD

☒ **Require status checks to pass before merging**
Require status checks to pass before merging.

- DevSecOps Demo Pipeline / build_and_test
- DevSecOps Demo Pipeline / security_scan

☐ **Require branches to be up to date before merging**
Require branches to be up to date before merging. This rule has some requirements that must be met before merging. The rule has some requirements that must be met before merging. The rule has some requirements that must be met before merging.

Resultado de la Configuración



Creación de PR

Permitida



Ejecución de Checks

Automática



Fusión de Código

Estrictamente bloqueada hasta obtener estado verde

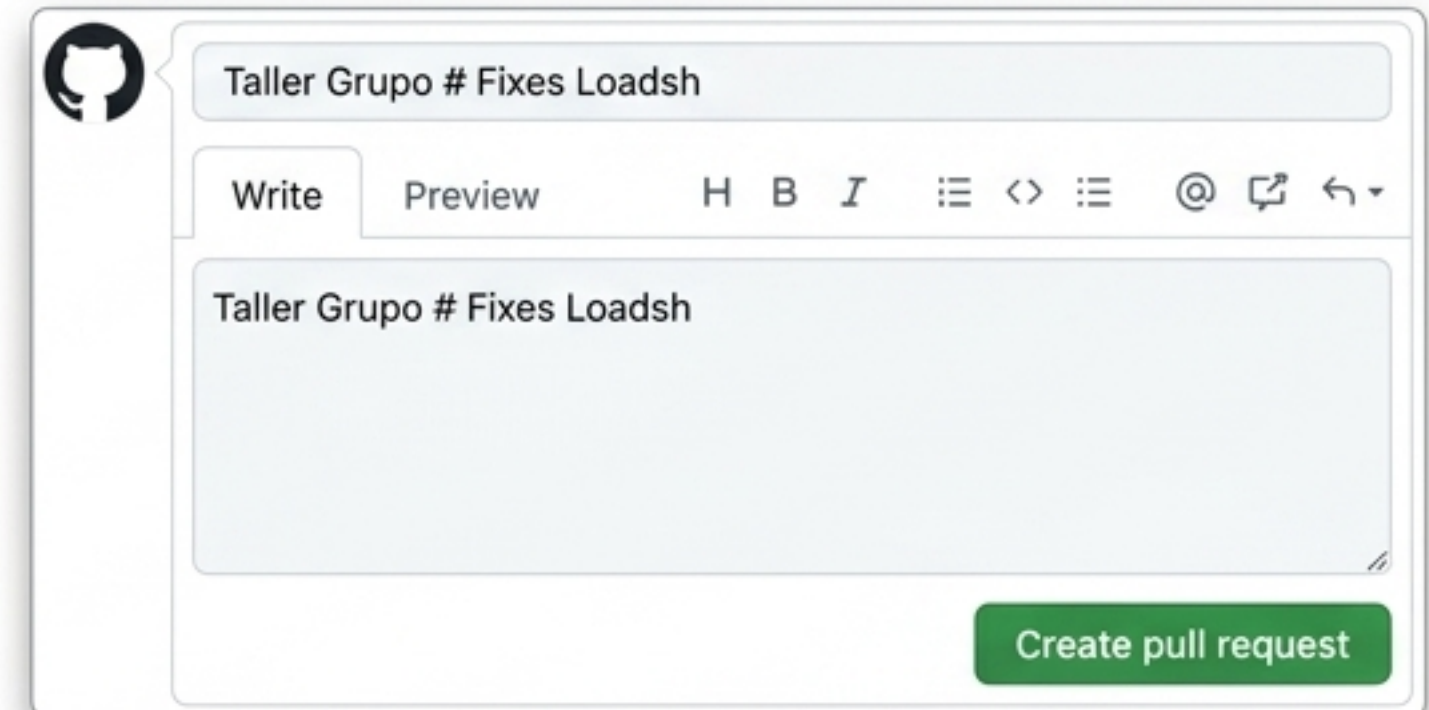
Prueba de Fuego: Introduciendo Vulnerabilidades

El Vector

```
< > {} package.json x
{
  "name": "sample_dev_sec_ops",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "jest",
    "lint": "eslint .",
    "audit": "npm audit --audit-level=high"
  },
  "dependencies": {
    --- "lodash": "4.17.21"
    +++ "lodash": "3.18.1",
    "react": "16.13.1",
    "react-dom": "16.13.1"
  },
  "devDependencies": {
    "jest": "^29.7.0"
  }
}
```

Contexto: Degradación intencional de la librería lodash a una versión conocida con vulnerabilidades críticas.

El Disparador



Acción: Creación del Pull Request desde la rama tallerI hacia main, introduciendo el código en el entorno.

El Pipeline Actúa



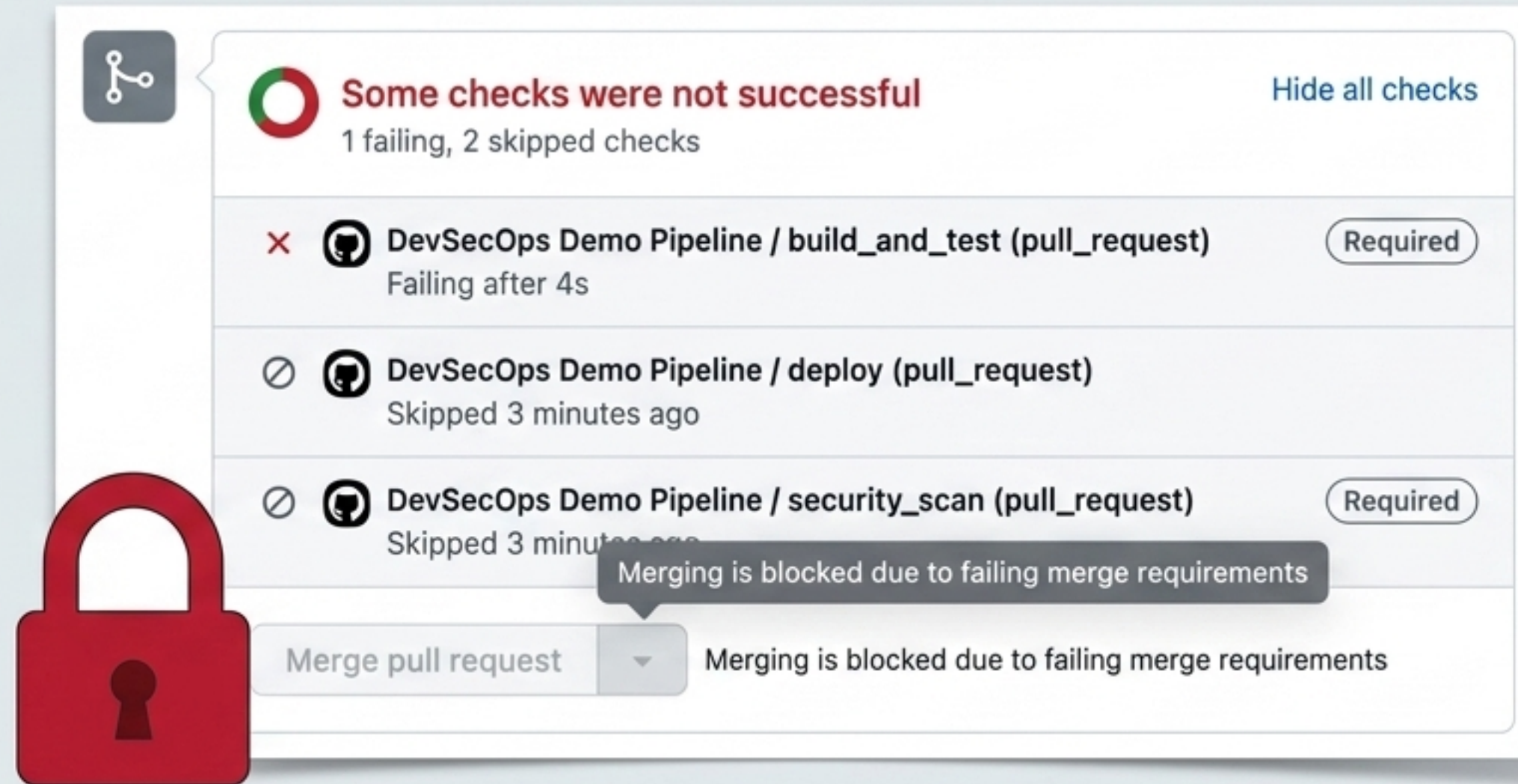
Some checks were not successful ×
1 successful, 1 failing, and 1 skipped checks

×	DevSecOps Demo Pipeline / security_scan (push) Failing after 15s	Details
✓	DevSecOps Demo Pipeline / build_and_test (push) Successful in 12s	Details
⊘	DevSecOps Demo Pipeline / deploy (push) Skipped	Details

Intercepción Automática

El escáner de dependencias detectó la versión **3.18.1** de **lodash** y abortó la operación antes de llegar a la fase de despliegue (deploy **skipped**).

Bloqueo Exitoso (Direct Branch)



 **Some checks were not successful** [Hide all checks](#)
1 failing, 2 skipped checks

	 DevSecOps Demo Pipeline / build_and_test (pull_request) Required Failing after 4s
	 DevSecOps Demo Pipeline / deploy (pull_request) Skipped 3 minutes ago
	 DevSecOps Demo Pipeline / security_scan (pull_request) Required Skipped 3 minutes ago

 Merge pull request  Merging is blocked due to failing merge requirements

Defensa Activa Confirmada.

El repositorio está blindado. La configuración aplicada en la etapa inicial ha inutilizado exitosamente el botón de Merge. La única forma de avanzar es remediando la vulnerabilidad del código.

Remediación: Asegurando el Código

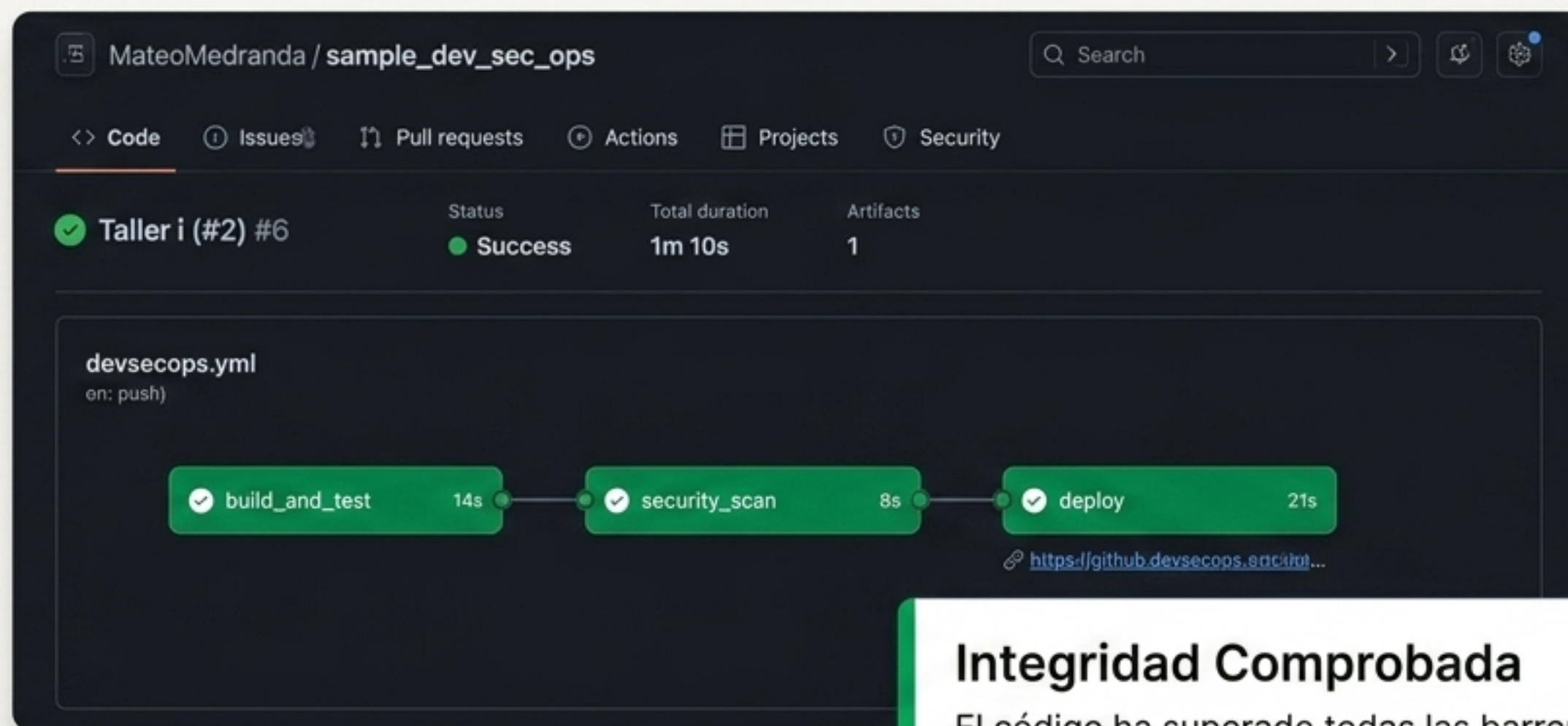
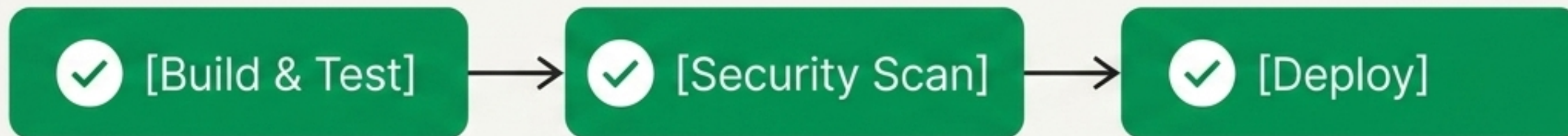


```
"dependencies": {  
  "lodash": "4.17.21"  
}
```

```
> Run npm audit --audit-level=high  
found 0 vulnerabilities
```

Al corregir la dependencia, el commit empuja una nueva revisión. El pipeline reinicia su ciclo de validación de forma autónoma.

Validación del Pipeline



Integridad Comprobada

El código ha superado todas las barreras lógicas. No existen conflictos de construcción, ni vulnerabilidades de nivel alto en las dependencias.

Luz Verde para el Merge



All checks have passed

1 skipped, 2 successful checks



No conflicts with base branch

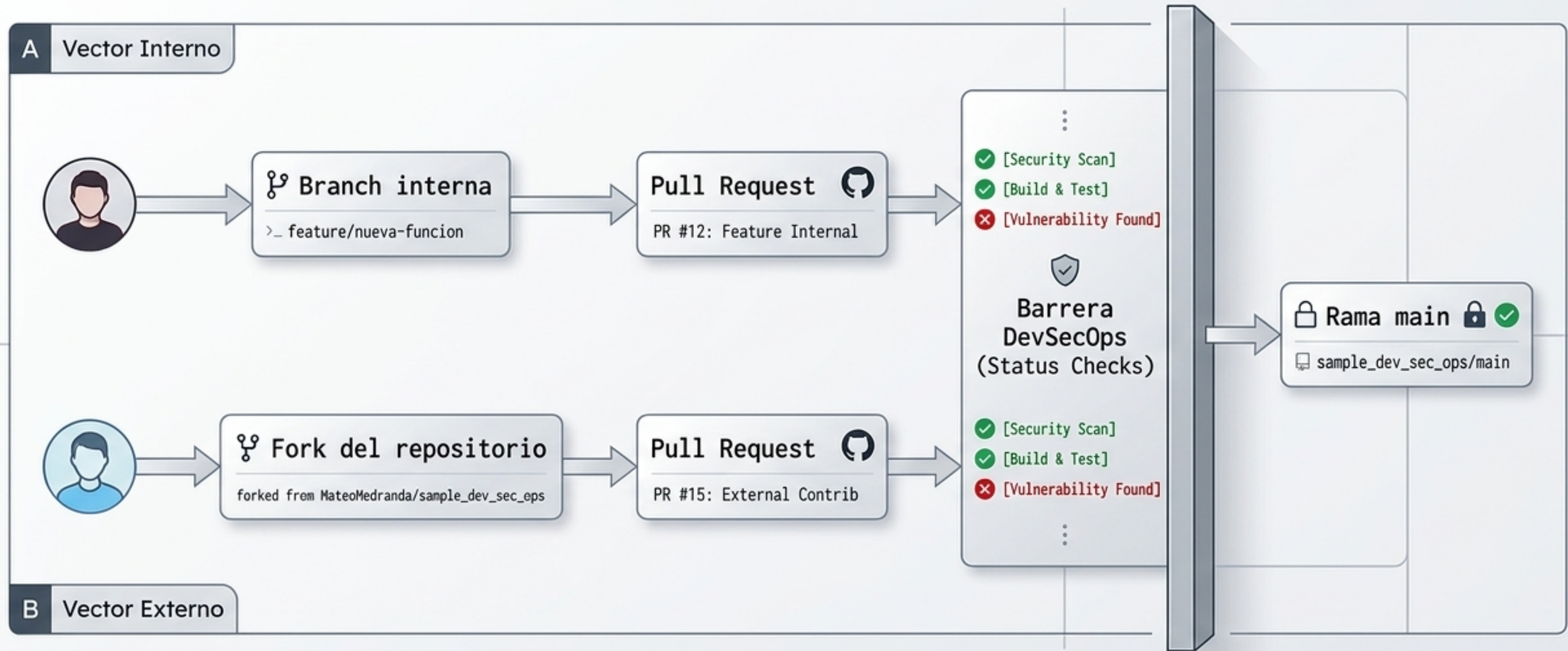
Merging can be performed automatically.

Merge pull request



Luz Verde. El ciclo primario está completo. La política de protección de rama y el pipeline operan en perfecta sinergia: detienen amenazas y facilitan integraciones seguras automáticamente.

Escenario Distribuido: Pull Request desde un Fork



En ecosistemas Open Source o equipos distribuidos, el código frecuentemente proviene de repositorios bifurcados (Forks). La arquitectura de seguridad debe ser agnóstica al origen del código.

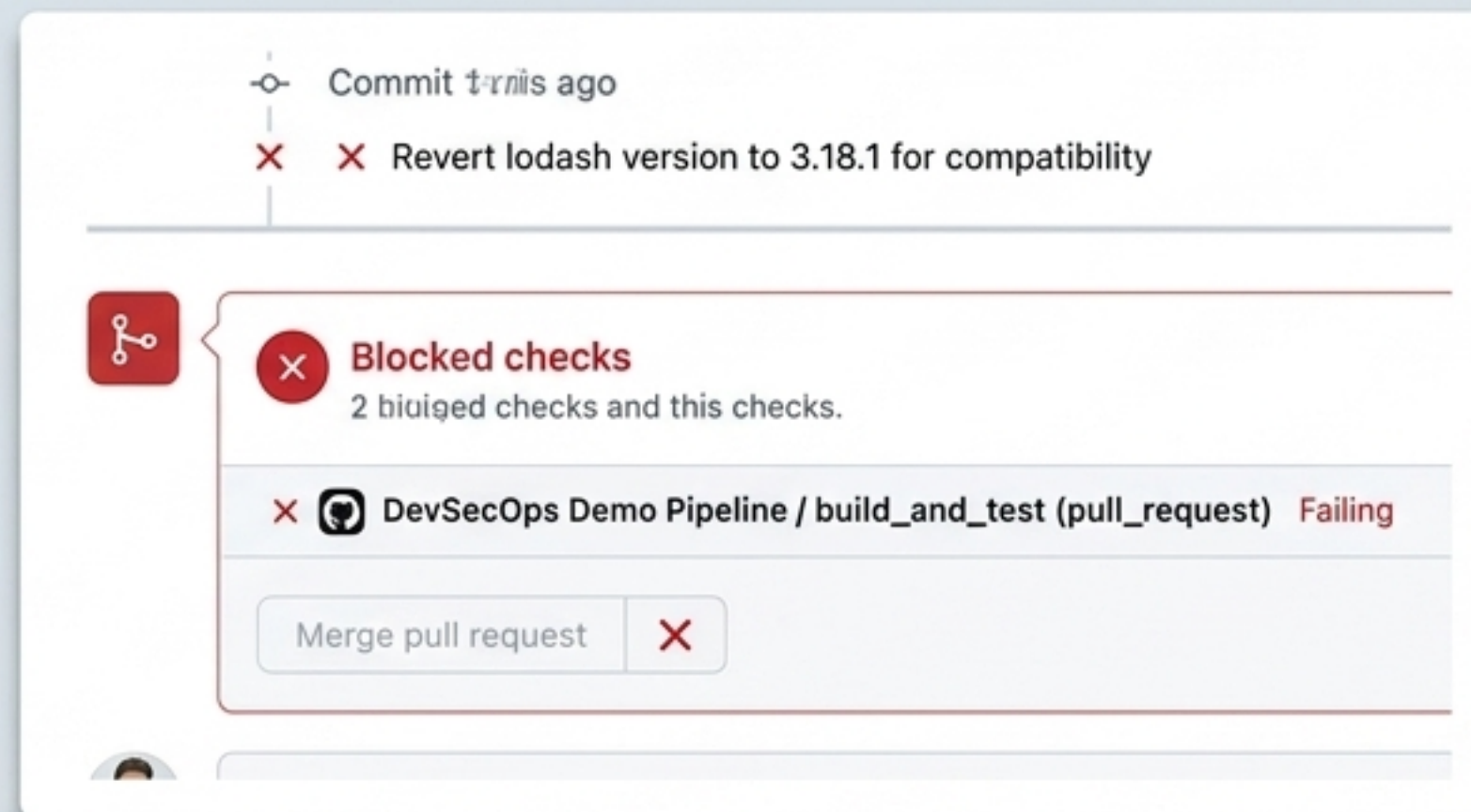
La Regla es Universal (Bloqueo en Fork)

El Contexto

LeoNarvaez2503 wants to merge 2 commits into
MateoMedranda:main from LeoNarvaez2503:main

Petición externa de un repositorio bifurcado.

El Resultado



La Regla es Universal. Las protecciones de rama de GitHub aplican rigurosamente los status checks a cualquier petición externa. El origen (Fork) no bypassa la seguridad.

Continuous Deployment (CD)

GitHub Pages

[GitHub Pages](#) is designed to host your pe

Build and deployment

Source

GitHub Actions ▾

Use a suggested workflow, [browse all wor](#)

mateomedranda.github.io

Pipeline DevSecOps

Esta página fue desplegada automáticamente solo si:

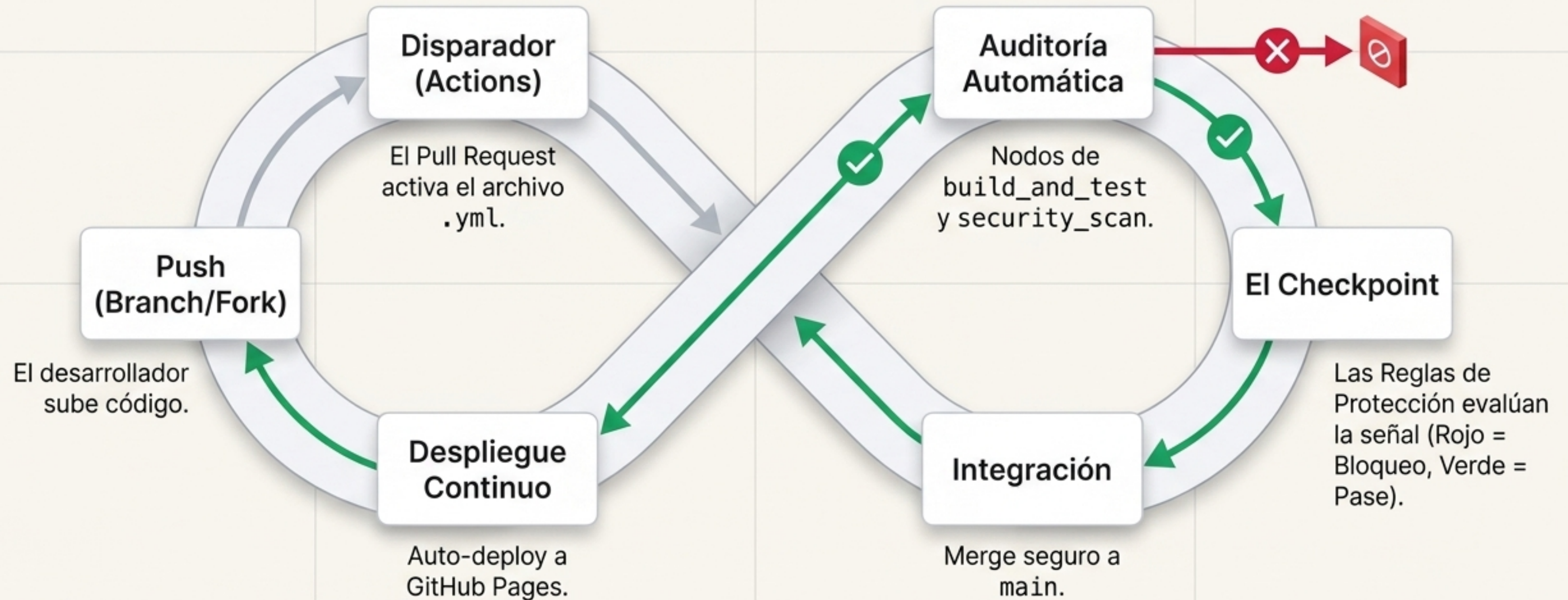
- ☒ (ok) Las pruebas pasaron (CI)
- ☒ (ok) El escaneo de dependencias no encontró vulnerabilidades altas (Sec)
- ☒ (ok) El deploy fue autorizado (CD)

Revisa el workflow en `.github/workflows/devsecops.yml`.

Takeaway

El despliegue continuo solo ocurre en la rama main, la cual está criptográficamente aislada de código defectuoso gracias a nuestra configuración.

El Flujo DevSecOps Definitivo



La seguridad dejó de ser un cuello de botella manual para convertirse en un **guardián automatizado y silencioso** del ciclo de vida del software.